



TOBB Ekonomi ve Teknoloji Üniversitesi
Bilgisayar Mühendisliği Bölümü
Elektrik ve Elektronik Mühendisliği Bölümü

4 Ağustos 2025
BİL 265/264/264L – Mantıksal Devre
Tasarımı ve Laboratuvarı
2024 – 2025 Öğretim Yılı
Yaz Dönemi
Final Sınavı

AÇIKLAMALAR:

1. Sınavı çözmeye başlamadan önce tüm açıklamaları ve soruları okuyun. Sınavda toplam 5 sayfa (3 adet A4 kağıdı), 2 soru var ve soruların toplam değeri **100** puandır. Bütün soruların değeri köşeli ayraç ile belirtilmiştir. Sınav süresi 150 dakikadır.
2. Sınav sırasında soru kabul edilmeyecektir.
3. Sınav esnasında internet ve tarayıcı kullanımı yasaktır. Bilgisayarda Xilinx Vivado programı dışında hiçbir program **KESİNLİKLE** açık olamaz.
4. İnternete bağlı olduğu veya herhangi bir tarayıcısı açık olduğu görülen kişilerin sınavları geçersiz sayılacak ve kopya olarak değerlendirilip gerekli işlemler yapılacaktır.
5. Sınav boyunca **sadece** bir yüzü dolu A4 sayfadan faydalanabilirsiniz. Bunun dışında her türlü araç/gereç ve kaynak kullanımı yasaktır.(hesap makinesi, akıllı saat, telefon, pdf dosyaları vb.)
6. Sonucu yanlış olan yanıtlar puan alamayabilir. Gidiş yolunun ayrıntılı gösterilmesi sorudan puan alınması için gereklidir ancak yeterli değildir. Açıklamasız kod yazmamaya özen göstermeniz alacağınız puanı artıracaktır.
7. <Projenizin bulunduğu dizin>\<Proje ismi>\<Proje ismi>.srcs\sources_1\new → dizininde yazdığınız “.v” uzantılı dosyaları bulabilirsiniz. Simülasyon dosyalarını ise aynı uzantıda .srcs’den sonra \sim_1 klasöründe bulabilirsiniz.
8. 9. talimata uyulmaması ve dosya isimlerinin yanlış yazılması durumlarında toplam puanınız üzerinden 20 puan kırılacaktır.
9. Dosya gönderimi için sorularda belirtilen “.v” uzantılı dosyalarınızı “isim_soyisim_numara_final” isimli bir klasöre attıktan sonra klasörü sıkıştırınız ve sınav sırasında gözetmenin getireceği USB veya Uzak’a yüklemeye hazır olacak şekilde bekleyiniz.

Sınavda Göndermeniz Gereken .v dosyaları: (Gönderim yapmak istemediğiniz soruları eklemek zorunda değilsiniz.)

- jk.v
- fibonacci.v
- kayan_yazmac.v
- kayan_yazmac_boru.v

1. [50 Puan] Fibonacci Sayacı (Verilog Davranışsal Modelleme)

Bu kısımda 2 adet modül oluşturacaksınız. Modüllerinize “jk”, “fibonacci” isimlerini verin (oluşacak dosyalar: jk.v, fibonacci.v). Modüllerinizi Verilog dilinde **davranışsal modelleme** ile gerçekleştirin.

a-) [15 puan] “jk” modülü:

jk modülünde JK flip-flop davranışını modellemeniz beklenmektedir. Saatin her yükselen kenarında J, K girişlerine uygun çıkışları vermeniz ve reset geldiğinde saat ile senkron şekilde devrenin durumunu sıfırlamanız gerekmektedir.

Hatırlatma:

$$J=0, K=0 \rightarrow Q(t+1) = Q(t)$$

$$J=0, K=1 \rightarrow Q(t+1) = 0$$

$$J=1, K=0 \rightarrow Q(t+1) = 1$$

$$J=1, K=1 \rightarrow Q(t+1) = Q'(t)$$

Yazacağınız modülün giriş ve çıkışları aşağıdaki gibidir:

Devrenin girişleri:

saat: 1 bitlik saat sinyali

reset: 1 bitlik reset sinyali (senkron)

J: 1 bitlik J giriş sinyali

K: 1 bitlik K giriş sinyali

Devrenin çıkışları:

Q: 1 bitlik Q çıkış sinyali

QN: 1 bitlik Q' çıkış sinyali

b-) [35 puan] “fibonacci” modülü:

fibonacci modülünde Fibonacci dizisini döngüsel olarak üreten 4-bitlik bir sayaç devresi tasarlanacaktır. Bu modülde a şıkında tasarladığınız **jk** modülünü alt modül-modüller olarak kullanmanız **zorunludur**. Modülün isterleri aşağıda verilmiştir.

- Eş zamanlı atamalar saatin yükselen kenarında yapılacaktır.
- Saat ile senkron şekilde “reset” sinyali geldiğinde modül başlangıç durumuna (0 değerine) dönmelidir.
- Sayaç, JK flip-floplar (jk modülü) kullanılarak tasarlanacaktır. (Fonksiyonel olarak kullanmanız **zorunludur**.)
- Sayacın çıktısı fibonacci dizisine uygun olarak ilerler ve 4 bit içinde kalan Fibonacci sayılarını döngüsel olarak gösterir. (0-1-1-2-3-5-8-13-0-...)
- Sayaç herhangi bir şekilde geçersiz (unused) bir duruma girerse, 2 sayısına gitmeli ve sayma işlemini kaldığı yerden doğru biçimde devam ettirmelidir.

Yazacağınız modülün giriş ve çıkışları aşağıdaki gibidir:

Devrenin girişleri:

saat: 1 bitlik saat sinyali

reset: 1 bitlik reset sinyali (senkron)

Devrenin çıkışları:

Q: 4 bitlik sayaç değeri çıkış sinyali

2. [50 Puan] Kayan Yazmaçlı Boru Hattı (Verilog Davranışsal Modelleme)

Bu kısımda 2 adet modül oluşturacaksınız. Modüllerinize “kayan_yazmac”, “kayan_yazmac_boru” isimlerini verin (oluşacak dosyalar: kayan_yazmac.v, kayan_yazmac_boru.v). Modüllerinizi Verilog dilinde **davranışsal modelleme** ile gerçekleştirin. Modüllerin birbiriyle ilişkisini ve giriş çıkışlarını verilen şemada görebilirsiniz.

Not: Bu kısımda parametrik tasarım yapamazsanız N=5 değeri için tasarlayın fakat bu durumda bir miktar puan kaybınız olacaktır.

a-) [25 puan] “kayan_yazmac” modülü:

kayan_yazmac modülünde N adet yazmaçta, giriş olarak gelen N bitlik sayı değerlerini tutarak her çevrimde bir sonraki yazmaca değer aktaran bir kaydırmalı yazmaç sistemi kurulacaktır. Modülün isterleri aşağıda verilmiştir.

- Eş zamanlı atamalar saatin yükselen kenarında yapılacaktır.
- Saat ile senkron şekilde “reset” sinyali geldiğinde modül başlangıç durumuna dönmelidir.
- Kaydırmalı yazmaç, “basla” sinyali mantık-1 olunca başlar, “miktar” girişi ile belirlenen kaydırma miktarı kadar çevrim boyunca anlık yüklenen sayı girişlerini bir sonraki yazmaca kaydırır ve kaydırma bitirdiği çevrim “hazır” sinyalini mantık-1 (bitirene kadar mantık-0) olarak sürer.
- “hazır” sinyalinin mantık-1 olduğu çevrim son kaydırma işleminin yapıldığı ve “sayi_cikis” çıkışından son yazmaçtaki değerin dışarı verildiği çevrimdir. Diğer çevrimlerde hazır sinyali mantık-0 olmalıdır. Yine, diğer çevrimlerde işlem bitmese bile “sayi_cikis” çıkışı son yazmaca bağlı olmalıdır.
- Yazmaçlar kaydırma aşamasındayken “basla” sinyalinin ne olduğu önemli değildir, sadece her işlem başlangıcında “1” olması yeterlidir. Kaydırma işlemleri bittikten sonra yeni bir “basla” sinyali gelene kadar yeni kaydırma işlemlerine başlanmaz.
- Saatin her yükselen kenarında sayi_giris değeri ilk yazmaca yüklenirken yazmaçlarda tutulan eski değerler bir sonraki yazmaca aktarılır.
- Modül, kaydırma işlemi bittikten sonraki çevrim -“hazır” sinyalinin “1” olduğu çevrimden sonraki çevrim- “basla” sinyali ile birlikte yeni girişleri alabilmelidir.
- “miktar” girişinin minimum N değeri kadar geleceğini ve kaydırma işlemi bitene kadar aynı değerde kalacağını varsayabilirsiniz.

Devrenin parametre girişi:

N: Kaç adet yazmaç olacağını belirleyen parametre girişi (varsayılan değer N=5)
($2 \leq N \leq 10$)

Devrenin girişleri:

saat: 1 bitlik saat sinyali

reset: 1 bitlik reset sinyali (senkron)

basla: 1 bitlik devrede işlemi başlatan giriş sinyali

sayi_giris: N bitlik, ilk yazmaca girecek sayı değeri giriş sinyali

miktar: 5 bitlik, kaç çevrim boyunca kaydırma işleminin yapılacağını belirten giriş sinyali

Devrenin çıkışları:

sayi_cikis: N bitlik, son yazmaçtan çıkacak sayı değeri çıkış sinyali

hazır: 1 bitlik, kaydırma işleminin bitip sayı çıkışının hazır olduğunu gösteren çıkış sinyali

Örnek 1:

N = 2

miktar = 6

basla = 1; sayi_giris = 1; → **yazmaclar = [X, X]; sayi_cikis = X; hazır = 0;** // ilk çevrim

basla = 0; sayi_giris = 3; → yazmaclar = [1, X]; sayi_cikis = X; hazır = 0;

basla = 0; sayi_giris = 2; → yazmaclar = [3, 1]; sayi_cikis = 1; hazır = 0;

basla = 0; sayi_giris = 1; → yazmaclar = [2, 3]; sayi_cikis = 3; hazır = 0;

basla = 0; sayi_giris = 1; → yazmaclar = [1, 2]; sayi_cikis = 2; hazır = 0;

basla = 0; sayi_giris = 0; → **yazmaclar = [1, 1]; sayi_cikis = 1; hazır = 1;** // 6. çevrim

→ **bir sonraki çevrim** → **hazır = 0;**

... // kaydırma bittikten sonra burada basla 0 olduğu müddetçe yeni işleme başlanmaz.

basla = 1 → yeni kaydırma işlemleri başlar.

Örnek 2:

N = 5

miktar = 10

basla = 1; sayi_giris = 12; → **yazmaclar = [X, X, X, X, X]; sayi_cikis = X; hazır = 0;**

basla = 0; sayi_giris = 7; → yazmaclar = [12, X, X, X, X]; sayi_cikis = X; hazır = 0;

basla = 0; sayi_giris = 31; → yazmaclar = [7, 12, X, X, X]; sayi_cikis = X; hazır = 0;

basla = 0; sayi_giris = 7; → yazmaclar = [31, 7, 12, X, X]; sayi_cikis = X; hazır = 0;

basla = 0; sayi_giris = 0; → yazmaclar = [7, 31, 7, 12, X]; sayi_cikis = X; hazır = 0;

basla = 0; sayi_giris = 14; → yazmaclar = [0, 7, 31, 7, 12]; sayi_cikis = 12; hazır = 0;

basla = 0; sayi_giris = 14; → yazmaclar = [14, 0, 7, 31, 7]; sayi_cikis = 7; hazır = 0;

basla = 0; sayi_giris = 14; → yazmaclar = [14, 14, 0, 7, 31]; sayi_cikis = 31; hazır = 0;

basla = 0; sayi_giris = 25; → yazmaclar = [14, 14, 14, 0, 7]; sayi_cikis = 7; hazır = 0;

basla = 0; sayi_giris = 13; → **yazmaclar = [25, 14, 14, 14, 0]; sayi_cikis = 0; hazır = 1;** // 10.

→ **bir sonraki çevrim** → **hazır = 0;**

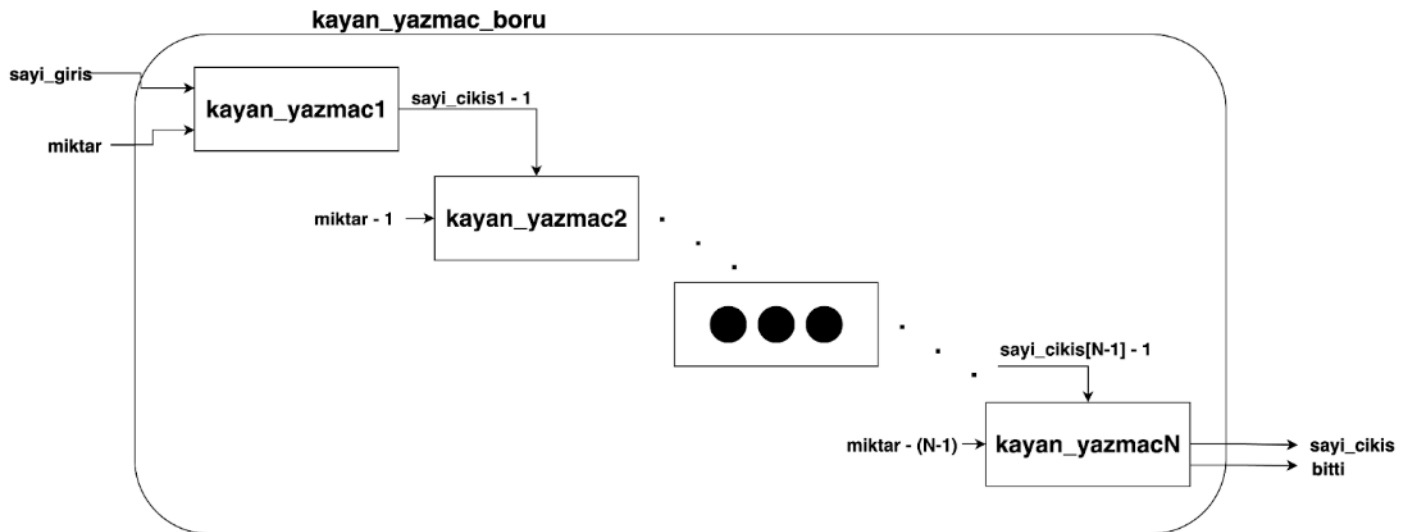
...

basla = 1 → yeni kaydırma işlemleri başlar.

b-) [25 puan] “kayan_yazmac_boru” modülü:

Bu modül, a şıkında tasarlamış olduğunuz kayan_yazmac modülünden N adet kullanarak art arda boru hatlı şekilde kaydırma işlemlerini gerçekleştirir.

Aşağıdaki şekilde boru hattı diyagramı verilmiştir.



Modülün isterleri aşağıdaki gibidir.

- Eş zamanlı atamalar saatin yükselen kenarında yapılacaktır.
- Saat ile senkron şekilde “reset” sinyali geldiğinde tüm modüller -alt modüller de dahil- başlangıç durumuna dönmelidir.
- N adet kayan yazmaç modülü birbirine boru hatlı şekilde bağlı çalışacaktır. Gelen girişler için başlangıçta 1. kayan yazmaç modülü başlayacak ve kaydirmayı bitirdiğinde 2. başlayacak, 2. bitirdiğinde 3. başlayacak, ..., (N-1). bitirdiğinde N. başlayacak ve N. çevrimden itibaren tüm modüller aynı anda çalışabilecektir. Boru hattı bu şekilde devam edecek ve farklı modüller farklı girişlerin değerlerini aynı anda işleyebilecektir.
- Boru hattı aşamalarındaki her kayan yazmaç bir önceki aşamada kullanılan kaydırma miktarının 1 eksiğini miktar girişinden alır. Kayan yazmaçlar için miktar değeri minimum N olacağından, miktarın 1 eksiğinin N kadar olduğu aşama ve sonraki aşamalarda miktar girişi N olarak sabit kalır. Örneğin; N = 5 olsun, boru hattına gelen miktar girişi -yani ilk kayan yazmaca gelen miktar girişi- 7 olsun. 1. kayan yazmaç 7 miktar değeri ile, 2. kayan yazmaç 6 miktar değeri ile, 3., 4. ve 5. kayan yazmaçlar ise miktar 5 değerleri ile çalışır.
- Boru hattı aşamalarındaki her kayan yazmaç bir önceki aşamadan çıkan sayı çıkışının 1 eksiğini sayi_giris girişinden alır. Alabileceği minimum değer 0 olabilir, negatif olamaz. Örneğin bir önceki aşamadan 1 ya da 0 değerleri sayı çıkışı olarak geldiyse sayı giriş değerini 0 olarak alır.
- Boru hattı devrenin genelinde kaydırma işlemini bitirdiğinde “bitti” çıkışı mantık-1, diğer durumlarda mantık-0 olur. Yani, kayan_yazmacN modülü her işlemini bitirdiğinde -“hazır” olduğunda- kayan_yazmac_boru modülünün “bitti” çıkışı 1 olmalıdır.
- Boru hattına gelen “miktar” girişinin boru hattı bitimine kadar sabit kalacağını varsayabilirsiniz.

Devrenin parametre girişi:

N: Hem kaç boru hattı aşaması olacağını hem de alt modüllerin her birinde kaç adet yazmaç olacağını belirleyen parametre girişi (varsayılan değer N=5)
($2 \leq N \leq 10$)

Devrenin girişleri:

saat: 1 bitlik saat sinyali

reset: 1 bitlik reset sinyali (senkron)

sayi_giris: N bitlik, ilk boru hattı aşamasındaki ilk yazmaca girecek sayı değeri giriş sinyali

miktar: 5 bitlik, 1. kayan yazmaçta kaç çevrim boyunca kaydırma işleminin yapılacağını belirten giriş sinyali

Devrenin çıkışları:

sayi_cikis: N bitlik, son boru hattı aşamasındaki son yazmaçtan çıkacak sayı değeri çıkış sinyali

bitti: 1 bitlik boru hattındaki kaydırma işleminin bittiğini belirten çıkış sinyali

Örnek: Başlangıçta N=2, miktar=6, sayi_giris=1 iken (sonraki çevrimlerde de bu girişlerin değişmediğini varsayarak ya da a şikkındaki örneğe bakarak) kayan_yazmac1 modülünden 6. çevrimde sayi_cikis=1 olarak çıkar ve kayan_yazmac2 modülü, 6. çevrimde sayi_giris=0 değeri ile başlar ve kayan_yazmac1 modülünden gelen sayi_cikis çıkışlarının 1 eksiğini sonraki çevrimlerde de almaya devam ederek boru hattına devam eder, 5 çevrim (miktar-1) boyunca kaydırma yapacağı için 10. çevrimde ilk hazır “1” çıktısını yani “bitti”yi verir, sonra yine boru hattına devam eder.